Soup - Full-Stack Engineer

Submission note | 11 November 2026

System and architecture

This mini experiment tracker uses Python 3.12, FastAPI, Pydantic v2, async SQLAlchemy/asyncpg, Alembic and PostgreSQL, with a Next.js App Router/React/TypeScript interface. FastAPI owns all business endpoints and SSE; Next.js serves the browser interface. Docker Compose starts web, API and PostgreSQL and applies migrations. Generated demo metrics exercise the actual ingestion/storage path; no ML training is claimed. The repository includes the original brief, one-page architecture, API contract, tests and raw results.

Identity, ordering and recovery

Events carry a client UUIDv4 event_id, metric, logical step, finite value and UTC timestamp. A versioned canonical payload defines identity. Retrying identical batch_id/content returns the original committed receipt; changed batch or event contents return 409. Identical event IDs across batches reuse their original sequence.

Writers lock the authenticated project then the scoped run. Event inserts, contiguous per-run seq allocation, the counter and receipt commit in one PostgreSQL transaction; success is returned only after commit. No process lock or nontransactional sequence supplies replay order. Highest (step,seq) wins for current values, so late lower-step events cannot replace newer logical values. An uncertain acknowledgement is resolved by retrying the same batch.

Snapshots use read-only REPEATABLE READ. SSE reads committed rows in pages of 250 from PostgreSQL, releasing sessions before sending. The browser uses fetch/ReadableStream with bearer Authorization, applies contiguous events idempotently and sends Last-Event-ID only after application. Replay covers the snapshot/subscription gap and process restart. Charts retain the last 2,000 events; current values include the full run. Initial chart reconstruction replays from 0; reconnection preserves its applied cursor.

Isolation, rotation and bounds

The bearer token selects project scope for every run lookup, mutation and replay. Only high-entropy token hashes are stored; rotation revokes the old hash transactionally. An independent 1-second watchdog checks existing subscriptions, including blocked replay/send paths. Already-buffered bytes cannot be recalled. Foreign and absent resources share 404 responses; protected responses use no-store. Keys stay in ephemeral browser memory, never URLs, localStorage or public environment variables. Reload requires key entry. Streams have bounded pages, a 5-second send deadline and 100-response/process admission; there is no unbounded subscriber queue.

Consistency and failure evidence

The recorded suites include 58 PostgreSQL-backed backend cases, 9 real Chromium E2E cases and 5 client cases. Browser checks compare actual chart points, history, winners, count and cursor with independent SQL after late/out-of-order/duplicate inputs. A cut partial frame 11 leaves cursor 10; replay restores exact 1-30. Other cases cover snapshot gaps, actual API/DB restarts, old-tenant frames after key switch, live rotation and negative cross-project REST/SSE access. PostgreSQL termination, deferred commit rejection, cancellation and lost HTTP receipts check rollback/retry behavior. A constrained unread TCP subscriber checks timeout cleanup and replay. Raw failures are retained; two historical Phase 2 compile/config captures are missing and explicitly unverified.

Load results and verdict

Each local workload used 50 runs producing 10 unique events/s/run in batches of 5, 10 seconds of warm-up and 60 seconds of measurement, plus 300 controlled exact retries. One Uvicorn process ran on Docker Desktop/WSL2 with 16 logical CPUs and 7.61 GiB shared VM memory, on an i5-13450HX laptop. Fixed-window throughput was 498.92 events/s without SSE and 498.17 with 50 SSE subscribers. All-request p95/p99 were 237.56/293.14 ms and 319.31/446.02 ms respectively. Each workload stored all 35,000 planned events and 7,000 receipts, with no HTTP failures/timeouts, missing acknowledged events or duplicate rows; SQL sequences were 1-700 per run and all snapshots matched stored winners. All 50 streams reached 700. One client resumed from 400 and recovered all 55 events acknowledged during its disconnect.

Verdict: SHIP for this measured local profile under the declared 1% tolerance. **The tolerance was self-defined, not specified by Soup. Exact 500 events/s was not achieved in either measured window, and production capacity remains unproven.** The finite runs do not establish an indefinite throughput floor or production headroom. Full commands, raw captures, audits and qualifications are indexed in docs/EVIDENCE.md and docs/SUBMISSION_AUDIT.md; final reproduction instructions are in docs/TESTING.md.

Limits and first production improvement

With 50 SSE consumers, sampled mean API CPU rose from 63% to 104% of one logical core, while observed PG connections reached 20, the configured pool ceiling. These indicate pressure, not a proven saturation point. First add transaction/pool-wait latency and CPU measurements, then profile SSE polling before changing concurrency or connection limits. Long soaks, many slow subscribers/aggregate socket buffers, large-history bootstrap, WAN/TLS/proxies, other browsers and disk/power loss are unverified. Literal loss of the PostgreSQL COMMIT acknowledgement was not injected. Generic errors and local timing/log probes do not prove side-channel absence. Unauthenticated local project bootstrap and lost key-rotation response recovery need a production access policy.

Surprise and concern - applicant reflection

I was surprised that the load generator itself caused the initial throughput problem rather than the backend; I had not expected the test client to be the limiting part. I am still concerned about how the system would behave with many slow SSE clients over a longer period. That scenario has not been properly tested: the existing evidence covers one deliberately constrained slow reader and 50 responsive subscribers for a short workload, not a sustained population of slow clients.

AI disclosure

Codex/ChatGPT assisted with architecture review, backend and frontend implementation, test generation, debugging, load tooling and documentation. Accepted choices include PostgreSQL-authoritative replay, transactional sequencing and fetch-based authenticated SSE. Generated code/tooling was revised after real failures, including timestamp serialization, a browser session race, stalled-send cleanup, DB connection-error handling and generator pool overhead. Cited commands were executed by Codex tools against real PostgreSQL/HTTP/Chromium; they are not applicant-run checks. The applicant selected FastAPI/Next.js, scope and dark UI, accepted phases and supplied the reflection above. No independent human code review or test execution is recorded. docs/AI_USAGE.md preserves provenance and remaining human review obligations. The employer's follow-up cancels the live discussion; this written submission is intended to stand alone.